

National University of Sciences & Technology (NUST)
School of Electrical Engineering and Computer Science (SEECS)
Faculty of Computing

CS 110: Fundamentals of Computer Programming
Assignment # 1

Assignment Date	28/09/2026	Due Date:	Sunday, 11/10/2025 (11:59 pm)	Marks: 20
CLO Attainment	[CLO-2]	Solve given real-world problem by applying appropriate programming concepts and techniques		

Assignment Title: **Relief Supply Distribution System**

Learning Objectives:

By completing this assignment, students will:

- Strengthen **problem-solving and logic design** skills.
- Experience **version control** using Git.
- Simulate **real-world decision making** with changing conditions.

Problem Description:

Imagine that you are developing a simple Relief Supply Distribution System for a district administration after a natural disaster such as flood.

The system receives a request from an affected area and decides how much relief should be sent. Supplies are limited, so the program must make sensible decisions and avoid exhausting stock too quickly.

Your group will develop a C++ program that:

- Calculates a need score for each relief request.
- Uses the score and other conditions to decide what action to take.
- Updates the available trucks and supplies after each action.
- Processes a given set of requests.
- Shows what happens when resources are sufficient and when they become insufficient.

Resources:

Following resources are declared for distribution:

- **Trucks:** number of trucks available for delivery.
- **Supply Packs:** food and medical packs available to load.

You can start with **5** Trucks and **120** Supply Packs. Later change these values to show program output when sufficient funds are available and when insufficient funds are available.

Program Input:

For every request, your program receives some input values from user. Your program must use these inputs to calculate a need score. Sample is given below:

Input	Possible Values
Damage Level	Low, Medium, High
Families Affected	10–200
Access Route	Open / Damaged
Flood Warning	Yes / No
Medical Need	Yes / No
Previous Aid Received	Yes / No

Possible Actions:

Based on the need score, your program can perform any of the following actions.

Action	Trucks	Supply Packs
A. Full Relief	2	50
B. Standard Relief	1	30
C. Emergency Kit Only	1	10
D. Defer / Monitor	0	0

Important: The program must never use more trucks or supply packs than are currently available.

Main Tasks:

Part A – Need Score Calculation

Develop your own formula for the need score. You may use any or all of the inputs above. You must explain why you gave different weight to different factors.

Example: ***[Not to use the same but rather design your own]***

Need Score = Damage Score + Family Score + Route Score + Flood Score + Medical Score − Aid Penalty

For instance: High damage = 30, Families ÷ 10, Damaged route = +10, Flood warning = +5, Medical need = +10, Previous aid = −5.

Part B – Decision / Action Rules

After calculating the score, decide which action to take. Your group must develop its own rules, for example using score thresholds combined with conditions such as remaining trucks, remaining supplies, medical need, or route status.

Your rules should be logical, clearly explained, and correctly implemented in C++. Think about what should happen when only enough resources remain for a smaller action.

Testing Your Program:

Use the following request set to test your program:

Request	Damage	Families	Route	Flood Warning	Medical Need	Previous Aid
R1 – Riverside Village	High	150	Damaged	Yes	Yes	No
R2 – Hilltop Settlement	Medium	60	Open	No	No	Yes
R3 – Lakeside Camp	High	90	Open	Yes	Yes	No
R4 – Market Town	Low	200	Open	No	No	No
R5 – Bridge Colony	High	40	Damaged	Yes	Yes	Yes

Required Testing:

Test Set 1 – Sufficient Resources: show the program processing the requests when resources are enough to respond to them.

Test Set 2 – Insufficient Resources: show the program's behavior when trucks and/or supply packs run out (e.g., by starting with fewer resources).

For each request, the output must show:

- Request information
- Calculated need score
- Selected action
- Trucks used and supply packs used
- Remaining trucks and remaining supply packs

At the end, display a summary of how many requests were successfully handled (and how many were deferred or could not be served).

C++ Requirements:

Demonstrate appropriate use of concepts covered in class, including:

- if, if-else, switch, and/or nested conditional statements
- Logical and relational operators
- Arithmetic and assignment operators

- Appropriate repetition structure
- Formatted output using cout with various escape sequences

Focus on clear logic and meaningful variable and function names.

Deliverables:

- C++ source code
- Flowchart(s)
- Test cases and program output for both sufficient and insufficient resources
- A short README/report containing: need score calculation; decision/action rules; explanation of why you designed them this way; test results; names of group members and their contributions; GitHub repository link; and a brief note on how (if at all) you used AI

The group may divide the work as it sees fit and highlight who did what in a table. However, for viva-voce every member must know everything about the project submission.

AI Use Policy:

You may use AI tools for understanding programming concepts, getting ideas for test cases, debugging your own code, and improving documentation.

You must not submit an AI-generated solution as your own work. If you use AI, briefly mention how in your README/report. Each group member must be able to explain the program and their contribution during the viva.

Assessment Rubric:

Each of the five criteria below is ranked on a scale of 1 to 4. The rank you receive is the number of marks for that criterion, so the maximum total is 5 × 4 = 20. Criteria 1–4 are marked for the group; Criterion 5 (Viva) is marked for each student individually.

How ranks are awarded:

- 4 – Exemplary: every statement in the descriptor is met.
- 3 – Proficient: the work is correct and complete overall, with minor gaps.
- 2 – Developing: the work is partly correct but has significant gaps or errors.
- 1 – Beginning: the work is attempted but mostly incorrect, unclear or incomplete.
- 0 – Nothing was submitted for that criterion.

FOCP Assignment 1

Criterion	4 – Exemplary	3 – Proficient	2 – Developing	1 – Beginning
1. Problem Analysis, Need Score & Decision Logic	• Formula uses at least 4 of the 6 inputs. • Every weight has a written reason linked to real-world impact. • Rules cover all four actions and check available trucks and packs. • Falls back to a smaller action when resources are short. • Hand-tracing the test requests shows no contradictions.	• Formula uses at least 4 inputs. • Reasons are given for most weights, but some are vague. • Rules cover all four actions and check resources. • At least one edge case is missed (e.g., enough trucks but not enough packs).	• Formula uses only 2–3 inputs, or copies the example without change. • Thresholds are given without reasons. • Some input combinations lead to no action or an illogical action.	• Formula or rules are missing, unexplained or illogical (e.g., low need gets Full Relief). • Available resources are not considered in the decision.
2. C++ Implementation & Programming Concepts	• Compiles and runs with no errors. • Uses appropriate decision control, logical and relational operators, arithmetic and assignment operators, a loop over requests • well commented code • meaningful names; consistent indentation.	• Compiles and gives correct output. • All required concepts appear, but one is weak • Naming or indentation is sometimes inconsistent. • nominal commenting.	• Compiles, but a required concept is missing. • Logic errors give wrong output for some requests. • Heavy code repetition. • bare minimum commenting	• Does not compile or crashes. • Most required concepts are absent, or output is hard-coded rather than calculated. • No commenting.
3. Testing, Resource Handling & Output	• Both test sets are shown. • Trucks and packs never go below zero or exceed what is available. • Output shows all items: request info, score, action, units used, remaining units, and the summary. • Insufficient set clearly shows a fallback or deferral with the reason printed. • Summary counts are correct.	• Both test sets are shown and resources are never exceeded. • Output has 5–6 of the required items, or the summary has a minor error. • Insufficient case is shown, but the reason for the fallback is not stated.	• Only one test set truly works, or the insufficient set never runs out of resources. • One or more required output items are missing. • Remaining values are inconsistent with the actions taken.	• Resources can go negative or be exceeded. • Most required output items are missing. • No summary is shown.
4. Flowchart & Documentation	• Flowchart matches the actual code and shows the decision branches and resource update. • README has every required item: formula, rules, rationale, results, member contributions, GitHub link, AI-use note. • Clearly written and well organised.	• Flowchart is present and mostly matches the code. • README has every required item, but one is thin (e.g., the rationale is one line).	• Flowchart is incomplete or does not match the code. • README is missing one or two required items.	• No flowchart • README is missing most required items or is not submitted.

Criterion	4 – Exemplary	3 – Proficient	2 – Developing	1 – Beginning
5. Viva & Individual Contribution (per student)	• Explains any part of the code and the reasons behind the design without help. • Correctly predicts the output for a changed input. • Contribution is specific and matches the README and repository history.	• Explains own part clearly and the general flow of the rest. • Traces a changed input with minor prompting. • Contribution is stated and believable.	• Explains own part only partially. • Needs significant prompting to trace a changed input. • Contribution is vague.	• Cannot explain the code or own contribution. • Answers suggest the work is not the student's own.

Scoring sheet:

Criterion	Rank (1–4)
1. Problem Analysis, Need Score & Decision Logic	
2. C++ Implementation & Programming Concepts	
3. Testing, Resource Handling & Output	
4. Flowchart / Pseudocode & Documentation	
5. Viva & Individual Contribution	
Total (out of 20)	

Important: There is no single correct formula or decision strategy. Ranks are based on whether your solution is logical, well-justified, correctly implemented, and handles both sufficient and insufficient resource situations.

Concise Deliverables:

The submission should contain following:

- Title Page (uploaded on LMS)
- Problem description and requirements
- Code attached as Annexure
- README/Report containing all information required
- Link to Git Repo
- Rubric + Scoring Sheet

All group members should submit the same document in LMS (Variations are not accepted).

Name the file using the given nomenclature:

FirstMemberName-SecondMemberName-ThirdMemberName-Assignment_1.docx.

No Late submissions are accepted. No submissions via email are excepted.

Plagiarism will be marked zero.